Building a Fact-Based Social Platform

A pre-deployment technical guide: tech stack, database design, authentication, the opinion/fact verification workflow, moderation, security, performance, and a free, production-ready infrastructure plan.

Prepared for: your text-only, opinion/fact social platform project
Scope: architecture & planning only — the fact-checking pipeline's internal logic is intentionally out of scope, to be designed separately
Every third-party service and free-tier limit cited here was verified by live web search in September 2026. Free-tier terms change often — re-check the linked pricing pages before you rely on a number in a launch decision.

Contents

0. How to Read This Document 3

What is deliberately out of scope . . . 3

The one honest tension in this whole plan . . . 3

Assumptions used to fill the gaps in your brief . . . 3

What we're deliberately skipping, and why . . . 4

1. Tech Stack Selection 5

1.1 Frontend — Next.js (React) . . . 5

1.2 Backend — Django + Django REST Framework . . . 5

1.3 Database — PostgreSQL . . . 6

1.4 Cache — Redis . . . 6

1.5 Why not GraphQL, microservices, or a custom search cluster . . . 6

2. System Architecture Overview 7

3. Database Schema Design 8

3.1 users . . . 8

3.2 posts . . . 8

3.3 follows, likes, comments, reports — the standard social graph . . . 9

3.4 moderation_actions — the audit trail . . . 9

3.5 Indexes worth calling out explicitly . . . 10

4. Authentication & Signup 11

4.1 What the signup form actually collects . . . 11

4.2 How the password is actually stored . . . 11

4.3 Email verification . . . 11

4.4 Sign-in methods: password, and “Continue with Google” . . . 12

4.5 Keeping bots off the signup form . . . 12

4.6 Staying logged in: access tokens, refresh tokens, and where they live . . . 12

5. Post Creation & the Opinion/Fact System 14

5.1 The submit flow . . . 14

5.2 The gap I filled: what “posted” means while a fact is being checked . . . 14

5.3 Displaying the confidence percentage . . . 15

5.4 The simplest background job setup that actually works . . . 15

6. Reporting System 16

6.1 The rule . . . 16

6.2 The threshold — a config value, not a guess . . . 16

6.3 Stopping the threshold from being gamed . . . 16
6.4 What happens when the threshold is crossed . . . 16
7. Fact-Check Pipeline Interface & Human Review Routing 17
7.1 The contract . . . 17
7.2 The routing decision . . . 17
7.3 Prior art worth knowing about . . . 17
8. Human Review / Moderation Panel 19
8.1 Why Django Admin specifically . . . 19
8.2 Two roles, not ten . . . 19
8.3 What a moderator actually sees . . . 19
8.4 An appeals path . . . 19
9. Core Social Features 20
9.1 Pagination: cursor-based, not page-number-based . . . 20
9.2 Computing the feed: fan-out-on-read, cached briefly . . . 20
9.3 Avoiding the N+1 query trap . . . 20
10. Search 21
11. Notifications 22
12. Security Checklist 23
13. Performance & Scalability 24
13.1 What actually makes this feel fast . . . 24
13.2 Load testing before you need to . . . 24
13.3 Being honest about the free-tier ceiling . . . 24
14. Free Infrastructure Map 25
15. DevOps & Deployment Pipeline 27
15.1 The whole pipeline, in one paragraph . . . 27
15.2 Environment variables and secrets . . . 27
15.3 Keeping the free backend warm . . . 27
16. Legal & Trust Basics 28
17. Testing 29
18. MVP Build Roadmap 30
19. Future Roadmap (Post-MVP) 31

0. How to Read This Document

This guide plans the engineering side of a text-only, fact/opinion social platform: what to build it with, how to store the data, how signup and login work, how a post moves from “just submitted” to “verified” or “flagged for a human,” and what infrastructure to run it on without paying for any of it. It is organised the way you'll actually build the project — starting at tech-stack selection, the same order you asked for.

What is deliberately out of scope

- **The fact-checking pipeline's internals.** You said you'll design that separately. This guide instead defines the *contract* around it — what goes in, what comes back, and what the rest of the system does with the result — so you can build everything else now and drop the real pipeline in later without re-architecting anything.
- **Images and video.** You said text-only for now. The database and storage choices below still leave room for media later (Section 14 covers the free object-storage option), but nothing is built for it yet.

The one honest tension in this whole plan

> **A NOTE BEFORE YOU START**
>
> “Free” and “Facebook-scale” pull in different directions, and it's worth separating them before you read another page. **Architecture quality** — clean data model, proper indexes, caching, a CDN, sensible pagination — costs nothing and can absolutely feel as fast and polished as a top platform for the traffic a new platform actually gets. **Raw capacity** — serving hundreds of millions of concurrent users — genuinely cannot be free anywhere, on any stack. Every recommendation below is chosen so the *patterns* match what large platforms use, while the *infrastructure* stays on real, currently-available free tiers. When you outgrow a free tier, it's because you have real usage — at that point the same architecture scales up by upgrading a plan, not by rebuilding.

Assumptions used to fill the gaps in your brief

You asked me to fill gaps with standard practice where your description didn't spell something out. Here is every place I did that, gathered in one place so you can override any of them before you build:

- **Baseline social features.** You described posting, opinion/fact classification, verification, and reporting in detail, but a platform “closely competitive” with Facebook/X/Instagram/LinkedIn implicitly needs follow, like, and comment functionality too. Section 9 covers these at a standard, unopinionated level.
- **Fact-post visibility while pending.** Your brief says a fact post is checked “before” it “will be posted,” but also that reports (which require a visible post) can trigger review. Section 5 explains the reading I used to reconcile this, and the alternative if you'd rather be stricter.
- **Report threshold numbers.** “Repeatedly reporting” isn't a specific count. Section 6 proposes a starting number and makes it a config value, not a hard-coded one, so you can tune it after launch.
- **Confidence threshold for auto-publishing.** Since the checking pipeline is a future component, Section 7 defines the threshold as a config value the pipeline's output plugs into, rather than guessing a number that belongs to a system that doesn't exist yet.
- **Sign-in methods.** You mentioned Google login by name; email/password plus Google OAuth is treated as the standard pair for a 2026 social app (Section 4).
- **Moderator roles.** “Human checking panels (kinda like an admin board)” is built as two permission tiers — Moderator and Senior Moderator/Admin — which is the smallest role split that still lets you separate day-to-day review from user/role management (Section 8).

What we're deliberately skipping, and why

You asked me to skip the complexity that big platforms carry for very small gains. Concretely, that means this guide will not ask you to set up any of the following — each one is a real thing Meta, X, or TikTok run, and each one would be a mistake at your current scale:

Big-platform technique	Why they need it	Why you don't, yet
Microservices architecture	Thousands of engineers need independent deploys	One small backend (a “monolith”) is faster to build, debug, and host — and free tiers bill per service, so fewer services is also cheaper.
Kubernetes	Orchestrating thousands of containers across data centers	Your free hosts (Section 14) already handle deploys, restarts, and scaling for a single service.
Custom Elasticsearch cluster	Sub-second search across billions of documents	PostgreSQL's built-in full-text search (Section 10) is free, zero extra infrastructure, and plenty fast for the post volumes a growing platform sees.
Multi-region active-active database	Sub-50ms reads for a global user base	A single-region database (Section 3) is simpler to reason about and is what your free database tier gives you anyway.
Custom fan-out-on-write timeline service	Precomputing feeds for hundreds of millions of users	Computing a feed at request time with a short cache (Section 9) is simpler and fast enough until you have a very large, very active user base.
GraphQL gateway	Letting many independent client teams query flexibly	A REST API (used throughout this guide) is simpler to secure, cache, and rate-limit for one frontend team.

One more framing note: **“comparable performance to top social media” is achievable, “identical scale” is not.** The architecture below will feel fast and behave correctly under real growth. It will not, on free infrastructure, serve Facebook's actual concurrent user count — nothing free does that anywhere.

1. Tech Stack Selection

This is the foundation everything else sits on, so it gets a firm recommendation rather than a menu of options. The table below is the whole decision at a glance; the reasoning for each row follows.

Layer	Choice	In one line
Frontend	Next.js (React)	Server-rendered React — fast, and posts are indexable/shareable like a real feed
Backend / API	Django + Django REST Framework	Batteries-included Python framework; its free admin panel becomes your moderation tool
Primary database	PostgreSQL (hosted on Neon)	Relational integrity for a graph of users, posts, follows, reports
Cache / rate limiting	Redis (hosted on Upstash)	Sessions, feed caching, and abuse-rate-limiting counters
Search (MVP)	PostgreSQL full-text search	Built into the database you already have — zero extra infrastructure
Object storage (future)	Cloudflare R2	Reserved for Phase 2 images/video; zero egress fees
Edge / CDN / security	Cloudflare (free plan)	DNS, CDN, DDoS protection, bot-check CAPTCHA, free TLS

1.1 Frontend — Next.js (React)

React is the recommendation because it has, by a wide margin, the largest ecosystem, the most tutorials, and the easiest path to hiring help later if this grows. Within React, Next.js specifically matters for a platform like this one because **individual posts need to be shareable and indexable** — when someone shares a link to a fact-checked post, you want a real page Google can crawl and a link preview that actually shows the claim and its verdict, not a blank shell that only fills in after JavaScript runs. That requires server-side rendering, which is what Next.js adds on top of plain React.

> **SIMPLER ALTERNATIVE**
>
> If Next.js's extra concepts (server components, the file-based router) feel like too much on day one, plain React with Vite is a legitimate simpler starting point — you lose the SEO/link-preview benefit until you migrate, but the component code you write is almost entirely reusable when you do.

1.2 Backend — Django + Django REST Framework

Django is recommended specifically because of what this project needs that a typical CRUD app doesn't: a **human review panel**. Django ships with a full, permission-aware admin interface generated automatically from your data models — list views, search, filters, bulk actions, and role-based access — for free, out of the box. Building an equivalent moderator dashboard by hand in a bare framework is routinely weeks of work. Section 8 shows exactly how this becomes your “human checking panel.”

Django also bundles an ORM, a migrations system, a user/permissions model, and a battle-tested auth system, so you're not assembling five separate libraries before you can write your first feature. Paired with

Django REST Framework (DRF), it exposes clean JSON endpoints for the Next.js frontend to call.

> **SIMPLER ALTERNATIVE**
>
> This does mean two languages in the codebase (Python backend, JavaScript/TypeScript frontend) instead of one. That's a real, normal pattern — frontend and backend are independent services talking over HTTP either way — not extra operational complexity. If you'd strongly prefer a single-language stack, Node.js with Express or NestJS is the alternative; you would then need a separate admin-panel library such as AdminJS or React Admin (both free and open source) to replace what Django gives you for free.

1.3 Database — PostgreSQL

The core data in a social platform — who follows whom, who liked what, which reports point at which post — is fundamentally a web of relationships. PostgreSQL is a relational database built exactly for that: it enforces referential integrity (you cannot orphan a like pointing at a deleted post), supports transactions (a report count and a status change update together or not at all), and still gives you flexible JSON storage (the `JSONB` type) for the one field here that genuinely benefits from it — the raw pipeline result.

> **DESIGN DECISION**
>
> Why not a NoSQL database like MongoDB? Document databases shine when your data doesn't have many relationships to enforce — a product catalog, a content-management system. A social graph is the opposite case: it's almost *all* relationships. Modelling follows/likes/reports in a document database means re-implementing, by hand, the referential integrity and multi-table transactions a relational database gives you automatically. PostgreSQL is also 100% free and open source with no paywalled tier of the database engine itself — you're only ever paying (or not paying) for hosting.

1.4 Cache — Redis

Redis is an in-memory key-value store used here for three specific jobs: holding session/rate-limit counters, caching a computed feed for a few seconds so repeat requests don't re-hit the database, and backing the simple background-job queue in Section 5. It's free and open source software; Section 14 covers a free hosted tier so you don't have to run a server for it yourself.

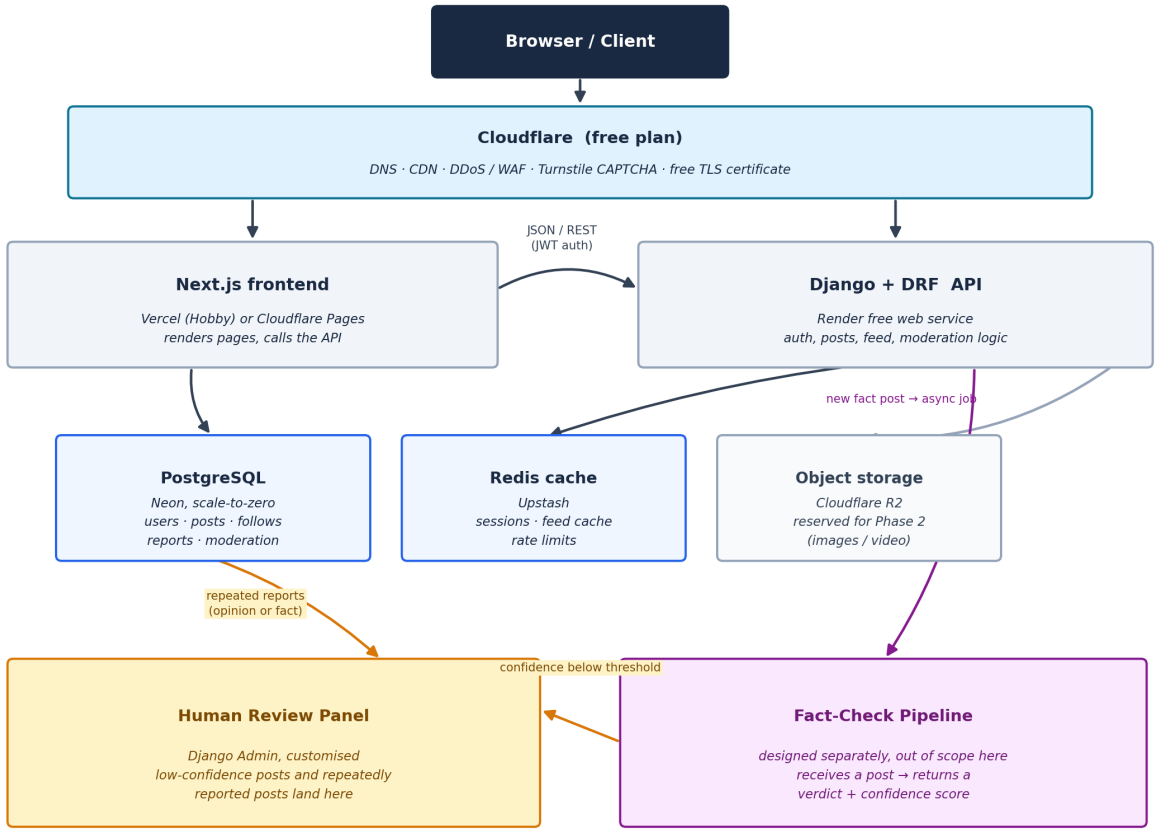
1.5 Why not GraphQL, microservices, or a custom search cluster

See the comparison table in Section 0 — the short version is that each of those solves a problem that only exists at a scale and team size you're not at yet, and each one adds real operational cost (more services, more moving parts, more free-tier line items) that a REST API and a single well-indexed Postgres database don't have.

2. System Architecture Overview

Everything in this guide is one small backend service, one frontend, one database, and one cache — deliberately. This is called a **monolith**, and for a team of one to a handful of developers it is the right choice: one thing to run, one thing to deploy, one place to look when something breaks. The diagram below is the whole system.

Figure 1. Request flow and component map



Two paths worth naming explicitly:

- **The request path** (top of the diagram): a browser talks only to Cloudflare, which forwards to either the Next.js frontend or, for API calls, straight to the Django backend. The backend is the only thing that talks to PostgreSQL, Redis, or object storage — the frontend never touches them directly.
- **The moderation path** (bottom of the diagram): whenever the backend decides a post needs a human — because the pipeline wasn't confident, or because it was reported enough times — that post's row is simply marked for review. There's no separate “queue service”; the queue *is* a filtered view over the posts table, which the Human Review Panel (Section 8) reads from directly.

> **WHY THIS IS FREE**
>
> Every box in this diagram maps to something with a genuine, currently-available free tier, verified in September 2026 (full detail and caveats in Section 14). Nothing here requires a credit card to start.

3. Database Schema Design

This is the schema the rest of the guide builds on. It's deliberately not maximal — every table here maps to a feature you described or a standard feature implied by “comparable to Facebook/X” (Section 0). Field names use `snake_case`, matching Django/Postgres convention.

3.1 users

Field	Type	Notes
id	UUID, primary key	UUIDs instead of auto-increment integers, so account IDs aren't guessable/enumerable
username	varchar(30), unique	indexed; shown publicly
email	varchar(254), unique	indexed; never shown publicly
password_hash	varchar	null if the account is OAuth-only — see Section 4
oauth_provider	varchar, nullable	e.g. 'google'; null for email/password accounts
oauth_subject_id	varchar, nullable	the provider's stable user ID, not their email
role	enum: user / moderator / senior_moderator	drives Section 8's permissions; default 'user'
is_email_verified	boolean	default false; see Section 4
bio	text, nullable	short profile text
avatar_url	varchar, nullable	reserved for Phase 2 — see Section 19
created_at	timestamptz	indexed for admin sorting

3.2 posts

Field	Type	Notes
id	UUID, primary key	
author_id	UUID, foreign key → users	indexed
content	text	post body; text-only per your spec
post_type	enum: opinion / fact	chosen by the author before posting — Section 5
status	enum: pending / verified_true / verified_false / under_review / removed	Section 5 and Figure 2
confidence_score	numeric(5,2), nullable	0.00–100.00; null until the pipeline responds
pipeline_result	JSONB, nullable	raw output from the future pipeline — kept flexible on purpose, see Section 7
flagged_reason	enum: pipeline / reports / both, nullable	why a post is under_review

Field	Type	Notes
report_count	integer, default 0	denormalised counter — see note below
moderated_by_id	UUID, foreign key → users, nullable	which moderator last acted on this post
moderated_at	timestamptz, nullable	
created_at	timestamptz	indexed — this is the main feed sort key

> **NOTE**
>
> `report_count` duplicates information you could also get by counting rows in the `reports` table — that's intentional. Checking "has this post crossed the report threshold" happens on every new report, so keeping a running counter on the post itself avoids a `COUNT(*)` query every time. Update it inside the same database transaction that inserts the report row, so the two never drift out of sync.

3.3 follows, likes, comments, reports — the standard social graph

Table	Key fields	Notes
follows	follower_id, following_id (composite primary key), created_at	both columns foreign keys → users; index both directions for fast "who follows me" and "who do I follow" lookups
likes	user_id, post_id (composite primary key), created_at	the composite key itself prevents double-liking at the database level
comments	id, post_id (FK), author_id (FK), content, created_at	kept simple for MVP — no nested replies yet, see Section 19
reports	id, post_id (FK), reporter_id (FK), reason, created_at	**unique constraint on (post_id, reporter_id)** — one report per user per post, enforced by the database, not just application code

3.4 moderation_actions — the audit trail

Every action a moderator takes gets logged here, separately from the post's current state. This exists for accountability: if a label is ever disputed, you can see exactly who changed it, when, and what it was before.

Field	Type	Notes
id	UUID, primary key	
post_id	UUID, foreign key → posts	indexed
moderator_id	UUID, foreign key → users	
action	enum: verified / marked_false / dismissed_report / removed / escalated	
notes	text, nullable	free-text rationale, optional but encouraged
created_at	timestamptz	

3.5 Indexes worth calling out explicitly

- `posts (status, created_at DESC)` — the main feed query filters by status and sorts by recency; a composite index serves both at once.
- `posts (author_id, created_at DESC)` — for profile pages.
- `GIN index on posts (to_tsvector('english', content))` — powers the Postgres full-text search in Section 10.
- `follows (follower_id)` and `follows (following_id)` — separately, since you query both directions.

4. Authentication & Signup

This section covers the two things you specifically asked for: what data signup stores, and which signup methods to offer. The design below is standard practice for a 2026 web app — nothing exotic, all of it free.

4.1 What the signup form actually collects

Field	Required?	Notes
Email	Yes	used for verification and password reset; never shown publicly
Username	Yes	public handle; checked for uniqueness as the user types
Password	Only for email/password signup	skipped entirely for Google OAuth signup
Cloudflare Turnstile token	Yes	invisible bot-check, not a visible puzzle — see 4.5

That's the whole form. Anything else — bio, avatar — belongs on a profile-completion step after the account exists, not on the signup form itself; every extra field on a signup form measurably reduces how many people finish signing up.

4.2 How the password is actually stored

The single most important rule: **the password itself is never stored, anywhere, ever** — not in the database, not in a log file. What's stored is the output of a one-way hashing function run on the password. Django's authentication system does this automatically; you don't write this code yourself, but you do choose the algorithm.

> **DESIGN DECISION**
>
> Use **Argon2id**, not Django's older PBKDF2 default. Argon2id won the Password Hashing Competition and is the current OWASP recommendation — it's deliberately slow and memory-hungry in a way that makes brute-forcing a stolen database expensive. In Django this is a one-line change: add `'django.contrib.auth.hashers.Argon2PasswordHasher'` as the first entry in `PASSWORD_HASHERS`, then `pip install argon2-cffi`. Both are free.

```
# settings.py
PASSWORD_HASHERS = [
    'django.contrib.auth.hashers.Argon2PasswordHasher',
    'django.contrib.auth.hashers.PBKDF2PasswordHasher',  # fallback for old hashes
]
```

4.3 Email verification

After signup, send a one-time verification link before the account can post. The flow:

1 Generate a random, single-use token and store it (with an expiry, e.g. 24 hours) alongside the new, still-`is_email_verified=false` user record.

2. Email a link containing that token via **Resend** (free tier: 3,000 emails/month, capped at 100/day — comfortably enough for an early platform's signups and password resets combined).
3. When the user clicks the link, the backend checks the token, flips `is_email_verified` to true, and deletes the token.
4. Let unverified users browse, but block posting and reporting until verification completes — this single gate removes a large fraction of throwaway bot signups for free, before Turnstile even matters.

> **WATCH OUT**
> A lot of older tutorials point to SendGrid's free tier for exactly this job. SendGrid discontinued its free plan in 2025 — it's now a paid-only trial. Resend is the current free equivalent; double-check resend.com/pricing before you build against these numbers, since email pricing has moved before.

4.4 Sign-in methods: password, and “Continue with Google”

You named Google login specifically, so here's both paths side by side.

	Email + password	Google OAuth
What's stored	Argon2id hash in password_hash	oauth_provider='google' + oauth_subject_id; password_hash stays null
Setup cost	None — built into Django	Free Google Cloud project + OAuth consent screen (just needs a Google account, no billing)
Library	Django's built-in auth	django-allauth (free, open source) handles the OAuth handshake
Email verification needed?	Yes (Section 4.3)	No — Google has already verified the email

The practical setup for Google sign-in: create a free project in Google Cloud Console, configure the OAuth consent screen, and generate a Client ID and Client Secret. `django-allauth` then handles the redirect-to-Google-and-back flow; on first login it creates a user row with no password and an oauth_subject_id, and on every later login it looks that ID up directly. This costs nothing at any realistic scale — Google doesn't charge for standard OAuth sign-in.

4.5 Keeping bots off the signup form

This matters more here than on a typical app: a platform whose core mechanic is *counting reports* to trigger human review is directly exposed to fake accounts being used to brigade a post into forced review. Two free, complementary defenses:

- **Cloudflare Turnstile** on the signup form — free, unlimited, and invisible to real users most of the time (no “click all the traffic lights” puzzles). It's a straightforward alternative to Google reCAPTCHA that doesn't share visitor data for ad targeting.
- **Rate limiting** on the signup and login endpoints themselves — e.g. via `django-ratelimit` (free, open source), capping signup attempts per IP address per hour. This is a second layer independent of Turnstile.

4.6 Staying logged in: access tokens, refresh tokens, and where they live

Since the frontend and backend are separate services talking over an API (Section 2), token-based auth fits better than Django's default cookie-session approach — it's also what you'd want if a mobile app ever gets

added later.

- Use `djangorestframework-simplejwt` (free, open source) to issue a short-lived **access token** (about 15 minutes) and a longer-lived **refresh token** (1–2 weeks).
- Keep the access token in memory on the frontend (a JS variable/React state) — **never in localStorage**. localStorage is readable by any script on the page, so a single XSS bug anywhere in the app would leak every logged-in user's token.
- Store the refresh token in an `httpOnly`, `Secure`, `SameSite=Lax` cookie — JavaScript can't read it even if XSS occurs, and the browser only sends it to your own domain.

> **NOTE**
>
> This access/refresh split is the standard trade-off between two real risks: a token in localStorage is vulnerable to XSS; a session cookie alone is more vulnerable to CSRF. Splitting the two roles — short-lived memory-only access token, httpOnly refresh cookie — is the common current answer to both at once, and every library involved is free.

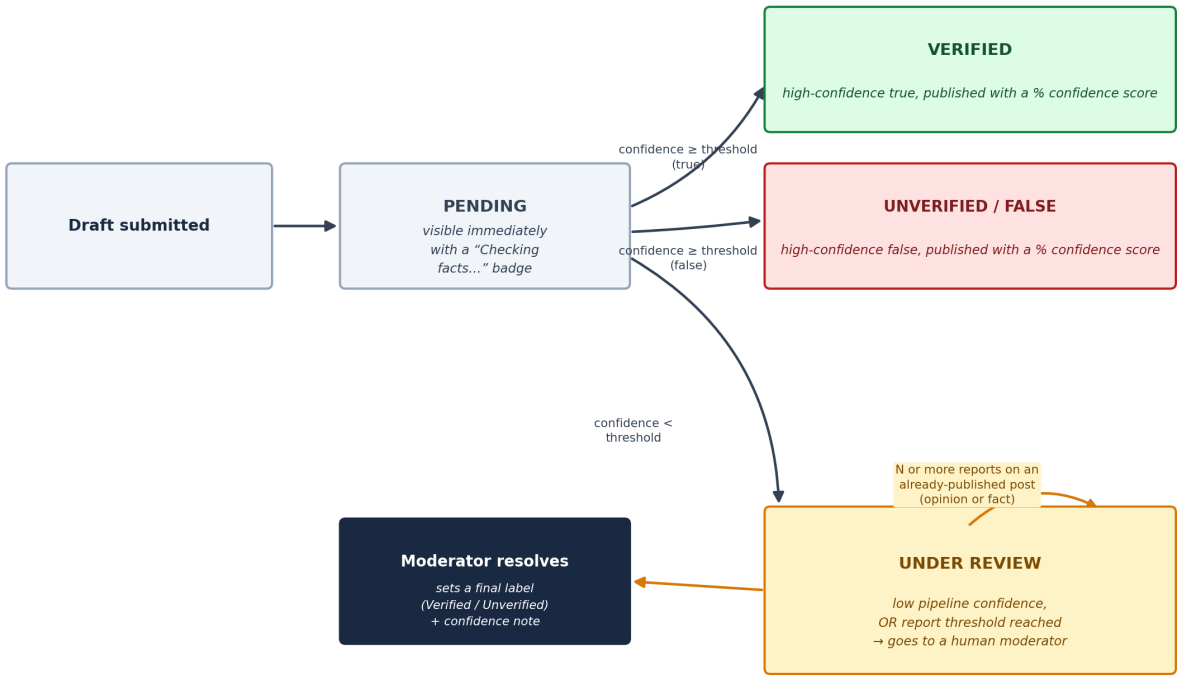
5. Post Creation & the Opinion/Fact System

This is the feature that makes your platform different from a generic social app, so it gets the most careful treatment in this guide.

5.1 The submit flow

1. The author writes their text and picks one of two options: **Opinion** or **Fact**.
2. **Opinion posts** are saved with `status='verified_true'`... actually, more precisely, opinions skip verification entirely — they're saved with a distinct status (`status='published'` in practice, since "verified/unverified" only makes sense for a factual claim) and go straight to the feed.
3. **Fact posts** are saved with `status='pending'` and appear in the feed *immediately*, carrying a "Checking facts…" badge instead of a verdict.
4. The post is handed to the fact-checking pipeline as a background job (Section 7 defines this contract precisely).
5. When the pipeline responds, the post's status updates in place — to Verified, Unverified/False, or Under Review — and whichever badge the reader sees changes automatically, with no reload needed if you poll for updates on that post (a simple, cheap approach for MVP; Section 11 covers upgrading this to push-based updates later).

Figure 2. Post status lifecycle



5.2 The gap I filled: what "posted" means while a fact is being checked

> **FILLING A GAP**

Your brief says a fact post is checked “before” it “will be posted,” which reads two ways: either the post stays completely invisible until the pipeline finishes, or “posted” means “finalised with a label” while the post itself is visible the whole time with a pending badge. I built the second reading, for two concrete reasons. First, it's the only reading that's consistent with reports being able to trigger review — a reader can't report a post they can't see, so reports on fact posts require the post to already be visible. Second, holding every fact post fully invisible until a (possibly backlogged) pipeline or human clears it doesn't scale on a volunteer/small-team moderation budget — it would mean new fact posts routinely sit unpublished for hours, which is a worse experience than a clearly-labelled “Checking facts…” state that's honest about its own uncertainty. **If you'd rather be stricter** — hide fact posts entirely until a verdict exists — that's a one-line change: filter `status='pending'` out of the public feed query in Section 9, and reports simply become impossible until the post first gets a verdict.

5.3 Displaying the confidence percentage

Once a verdict exists, show it plainly rather than burying it — this is the platform's core promise to readers. A simple, honest format:

- **Verified ✔ (94% confidence)** — green treatment
- **Unverified / False ✘ (88% confidence)** — red treatment
- **Under Review** — amber treatment, no percentage shown (a low or inconclusive score displayed as a bare number reads as more precise than it is; “under human review” communicates the actual state better than e.g. “52% confidence” would)

5.4 The simplest background job setup that actually works

Fact posts need *something* to happen asynchronously — you don't want the API request that creates a post to hang while a (future, possibly slow) pipeline runs. The common reflex here is to reach for Celery with a Redis broker. That's the right answer at real scale, but it's also a whole second system to run, monitor, and keep alive — unnecessary complexity this early, per your own instruction to skip what only pays off later.

DESIGN DECISION

For MVP, use a **database-backed queue** instead: a lightweight worker process (a simple Django management command run on a schedule, or a tiny loop) polls for `posts` rows where `status='pending'`, sends each to the pipeline, and writes back the result. No new infrastructure, no new free-tier account to manage — the queue is just rows in the database you already have. When volume genuinely outgrows polling (Section 13 covers what that looks like), swap in Celery + the Redis you already have running for caching; the pipeline's input/output contract in Section 7 doesn't change either way.

6. Reporting System

Reports apply to both opinion and fact posts, exactly as you specified — misleading opinions and wrong facts are both worth flagging, and the routing logic (Section 7) treats a report on either type the same way.

6.1 The rule

One report per user per post, enforced at the database level by the unique constraint on `(post_id, reporter_id)` defined in Section 3 — not just checked in application code, which matters if two report requests ever race each other.

6.2 The threshold — a config value, not a guess

> **FILLING A GAP**
>
> Your brief says “people repeatedly reporting a single post” without a number. Rather than invent one, treat it as a setting: `REPORT_THRESHOLD = 5` (`reports within a 24-hour window`) is a reasonable starting point — low enough that a genuinely bad post gets caught quickly, high enough that one annoyed user can't single-handedly force anything into review. Put it in Django's settings, not hard-coded in a function, so you can tune it after watching real reporting behaviour for a few weeks.

6.3 Stopping the threshold from being gamed

A raw report count is an obvious target for coordinated brigading — a group of accounts dogpiling a post they simply disagree with, not one that's actually false or against the rules. Two cheap, standard mitigations:

- **Weight or ignore reports from very new accounts.** A simple version: reports from accounts younger than, say, 24 hours don't count toward the threshold until the account has done something else on the platform first. This alone blocks the laziest version of “create 5 accounts, report one post.”
- **Rate-limit reporting itself** the same way Section 4.5 rate-limits signups — a cap on reports per user per hour, via the Redis-backed rate limiter you already have running.

6.4 What happens when the threshold is crossed

The report count crossing the threshold sets `flagged_reason='reports'` (or `'both'` if the pipeline had already flagged it for low confidence) and moves the post's `status` to `under_review`. The post stays visible — removing it outright on report volume alone, before any human looks at it, would let brigading achieve exactly what it's trying to achieve. Section 7 covers the full routing logic, and Section 8 covers what a moderator does with it from there.

7. Fact-Check Pipeline Interface & Human Review Routing

You'll design the pipeline itself later. What matters right now is the **contract** around it — fixed inputs and outputs so every other piece of this system (the post status, the confidence badge, the review queue) can be built today and never need to change when the real pipeline arrives.

7.1 The contract

	Field	Type
Pipeline receives	post_id	UUID
	content	text (the claim being checked)
Pipeline returns	verdict	enum: true / false / inconclusive
	confidence_score	numeric, 0–100
	raw_result	arbitrary JSON — sources, reasoning, anything the pipeline wants to attach

`raw_result` lands directly in the `pipeline_result` JSONB column from Section 3 — deliberately schema-less, so the pipeline's internal design can change freely later without a database migration.

7.2 The routing decision

This is plain logic, run once whenever the pipeline responds or a report count changes:

```
if post.post_type == 'opinion':
    status = 'published'          # opinions are never pipeline-checked

elif pipeline_result.confidence_score >= CONFIDENCE_THRESHOLD:
    status = 'verified_true' if pipeline_result.verdict == 'true' else 'verified_false'

else:
    status = 'under_review'
    flagged_reason = 'pipeline'

# independently, on every new report:
if post.report_count >= REPORT_THRESHOLD:
    status = 'under_review'
    flagged_reason = 'both' if flagged_reason == 'pipeline' else 'reports'
```

> **FILLING A GAP**
>
> `CONFIDENCE_THRESHOLD` is left as a config value (a sensible starting point is 80–85%) rather than a fixed number, for the same reason as the report threshold in Section 6: it belongs to a pipeline that doesn't exist yet, and you'll want to tune it once you can see real confidence-score distributions.

7.3 Prior art worth knowing about

X's Community Notes (formerly Birdwatch) is the most visible real-world example of crowd fact-checking on a social platform, and it's worth understanding even though it solves a different problem than yours. Rather than a moderation team, it uses a *bridging-based ranking algorithm*: notes get surfaced only when people who

normally disagree with each other both rate the note as helpful, which is specifically designed to resist one-sided brigading.

> **NOTE**
>
> That's a genuinely more complex system than what you asked for — it requires a whole separate rating layer, a contributor-reputation model, and a purpose-built ranking algorithm. Your brief describes a dedicated human review panel instead, which is simpler to build and ship first. Section 19 (Future Roadmap) notes bridging-style crowd rating as a plausible later addition, once you have enough active users for “cross-perspective agreement” to mean anything statistically — it doesn't help with only a handful of moderators.

8. Human Review / Moderation Panel

This is the “human checking panel (kinda like an admin board)” from your brief. The fast, free way to build it is to lean directly on Django's built-in admin interface rather than building a moderator dashboard from scratch as a separate frontend app.

8.1 Why Django Admin specifically

Django generates a complete, working CRUD interface — list views, search, filtering, sorting, bulk actions, and permission checks — directly from your data models, with no additional frontend code. Pointed at the `posts` table, a few dozen lines of configuration produce a usable review queue: filter to `status='under_review'`, sort by report count or by how long it's been waiting, and let a moderator act on it — all before you've written a single line of custom UI. This is squarely in the spirit of “skip the complex route for a small gain”: a hand-built moderator dashboard is real, ongoing frontend work for something Django gives you for free.

8.2 Two roles, not ten

Role	Can do	Cannot do
Moderator	Review flagged posts; mark Verified / Unverified / dismiss a report	Change another user's account role; delete users
Senior Moderator / Admin	Everything a Moderator can, plus: manage moderator accounts, adjust REPORT_THRESHOLD / CONFIDENCE_THRESHOLD, view the full audit log	—

Implemented as two Django Groups with different model-level permissions — no custom permissions system to build, since this is exactly what Django's auth app is for.

8.3 What a moderator actually sees

A custom admin view, filtered to the review queue, showing for each post: the content, its type (opinion/fact), `flagged_reason`, the pipeline's confidence score if one exists, the report count, and the reports themselves with their stated reasons. Bulk actions handle the common cases in one click across multiple posts at once: **Mark Verified**, **Mark Unverified/False**, **Dismiss Reports** (post stays as-is), and **Remove Post** (for content that breaks other rules entirely — spam, harassment — a separate concern from fact-accuracy that any real platform needs a path for). Every action writes a row to `moderation_actions` (Section 3.4) automatically.

8.4 An appeals path

Because an incorrect “false” label carries real reputational weight for whoever wrote the post, give authors a way to contest one: a simple “Request another look” action on their own post that re-adds it to the review queue with a flag noting it's an appeal, ideally routed to a *different* moderator than the one who made the original call where your moderator headcount allows it. This is a small addition now and a meaningful trust signal later — see Section 16 for why it matters beyond user experience.

9. Core Social Features

Follow, like, and comment aren't in your original description, but they're implied by wanting to be “closely competitive” with Facebook, X, Instagram, and LinkedIn (Section 0). The schema for all three is already in Section 3.3; this section covers how the feed itself is built — the part that actually determines whether the app *feels* fast.

9.1 Pagination: cursor-based, not page-number-based

The instinctive approach — “give me posts 21 through 40” (`OFFSET 20 LIMIT 20`) — gets slower as the offset grows, because the database still has to scan through and discard everything before it, and it produces duplicate or skipped posts if new ones arrive while someone's scrolling.

> **DESIGN DECISION**
>
> Use **cursor-based pagination** instead: “give me 20 posts created before this exact timestamp/ID.” Paired with the `posts (status, created_at DESC)` index from Section 3.5, this stays fast regardless of how far someone scrolls, and it's the same technique X, Instagram, and LinkedIn all use for their own feeds — not a compromise, the actual industry-standard approach.

9.2 Computing the feed: fan-out-on-read, cached briefly

There are two classic ways to build a “posts from people I follow” feed:

- **Fan-out-on-write**: the moment someone posts, pre-compute and push that post into every follower's personal, precomputed timeline. This is what Twitter-scale infrastructure actually does, because it makes reading a feed nearly instant — at the cost of a genuinely complex write path and dedicated infrastructure to manage it.
- **Fan-out-on-read**: compute the feed at request time — “posts from everyone I follow, newest first” — directly against the indexed `posts` table.

Fan-out-on-read is the right choice here: it's a single indexed query, it needs no extra infrastructure, and it's exactly correct at any scale a free-tier database will realistically see. Cache each user's computed feed in Redis for a short TTL (20–60 seconds) so a burst of refresh clicks doesn't re-hit Postgres every time — that alone gets most of fan-out-on-write's speed benefit without any of its complexity.

9.3 Avoiding the N+1 query trap

A feed of 20 posts naively showing each author's name, follower-status, and like-count can silently turn into dozens of extra database round-trips — one per post, per related field. Django's ORM avoids this for free if you ask it to: `select_related()` for the post's single author, and `prefetch_related()` for its many likes/comments, both fetch everything in one or two queries instead of one-plus-N. This single habit is often the biggest, cheapest performance win available in a Django app.

10. Search

Start with what you already have running: PostgreSQL includes real full-text search (`tsvector`/`tsquery`), with ranking, stemming (“running” matches “run”), and stop-word handling built in. Paired with the GIN index from Section 3.5, this searches post content directly — no new service, no new free-tier account, nothing extra to keep alive.

> **DESIGN DECISION**
>
> Upgrade to a dedicated search engine only once Postgres search genuinely isn't enough — typically when you want typo-tolerance (“JavaScrpt” still finding “JavaScript”) or sub-50-millisecond results at meaningfully larger volume. At that point, **Meilisearch** is the right free option: fully open source (MIT licensed), free with no feature limits when self-hosted, and dramatically simpler to run than standing up an Elasticsearch cluster — the kind of “complex route for a small gain” Section 0 flags for a platform at your stage.

11. Notifications

For MVP, a plain database-backed approach: a `notifications` table (recipient, type, related post, read/unread, timestamp) that the frontend polls every 15–30 seconds while the app is open. It's not real-time, but it's simple, needs no new infrastructure, and is genuinely unnoticeable to a user at that polling interval.

When true real-time matters more — for example, updating a fact post's badge the instant a verdict lands — upgrade to **Django Channels** (free, open source) for WebSocket push. This is a reasonable Phase 2 addition once the polling approach's limits actually show up in practice, not before.

12. Security Checklist

None of this requires a paid tool — every mitigation below is either free software or a free service tier already named elsewhere in this guide.

Risk	Mitigation	Where it's from
SQL injection	Use the Django ORM's parameterised queries everywhere; never build raw SQL by string-concatenating user input	Built into Django
Cross-site scripting (XSS)	React escapes rendered text by default; never use dangerouslySetInnerHTML on user content without sanitising it first (DOMPurify)	Built into React + DOMPurify (free, open source)
Cross-site request forgery (CSRF)	Low risk by design here — the JWT access token travels in an Authorization header, not an auto-sent cookie (Section 4.6); Django's CSRF protection still covers the refresh-token cookie endpoint	Built into Django
Brute-force login / signup	Rate limiting per IP on auth endpoints	django-ratelimit (free)
Bot signups / fake accounts	Invisible CAPTCHA on signup	Cloudflare Turnstile (free, unlimited)
Report-count manipulation	New-account report weighting + report rate limiting	Section 6.3
DDoS / scraping abuse	Edge-level protection in front of everything	Cloudflare free plan
Stolen database, exposed passwords	Argon2id hashing, never plaintext, ever	Section 4.2
Leaked secrets in source control	API keys and DB credentials in environment variables, never committed; `.env` files in .gitignore	Standard practice, free
Known library vulnerabilities	Automated dependency-update alerts	GitHub Dependabot (free)
Traffic not encrypted	HTTPS enforced everywhere	Free TLS certificates via Cloudflare / Let's Encrypt, automatic on every host in Section 14

13. Performance & Scalability

13.1 What actually makes this feel fast

- **The indexes from Section 3.5** — far more of a platform's perceived speed comes from correct indexing than from any infrastructure choice.
- **select_related / prefetch_related everywhere a feed touches related data** (Section 9.3).
- **A short Redis cache** on computed feeds and any other expensive, frequently-repeated read.
- **Cloudflare's CDN** sitting in front of the frontend, serving static assets from locations near the reader instead of a single origin server every time.

13.2 Load testing before you need to

k6 (free, open source) can simulate hundreds of concurrent users hitting your API and will surface slow endpoints and missing indexes before real users do. Worth a run before any public launch moment you're expecting a traffic spike for — a launch post, a press mention — rather than the first time you find out under load.

13.3 Being honest about the free-tier ceiling

This is the practical edge of the tension named in Section 0. Concretely, on the free stack in this guide:

- The architecture (indexing, caching, cursor pagination, a CDN) is the same shape used at real scale, so nothing here needs to be *rebuilt* as you grow — only *resourced* differently.
- The *infrastructure* has real ceilings: Render's free web service is a single small instance that sleeps after 15 minutes of inactivity; Neon's free database caps out at 0.5 GB storage and 100 compute-hours a month (Section 14 has the full picture). These comfortably cover early users and testing. They are not where a platform with thousands of daily active users stays for free.
- When you outgrow them, the upgrade path is a plan change, not a rewrite — e.g. Render's paid tier removes the sleep behaviour for about $7/month with the same codebase running unmodified.

14. Free Infrastructure Map

Every entry below was checked by live web search in September 2026, not pulled from memory — free tiers change often enough that older tutorials and even older parts of my own training data get this wrong. Re-verify anything you're about to build a launch decision around; the note column flags the catches that matter most.

Component	Free service	Free-tier limit (Sept 2026)	Watch out for
Database	Neon (PostgreSQL)	0.5 GB storage, 100 compute-hrs/mo, scale-to-zero	Suspends after 5 min idle, wakes in ~500ms — fine for production, unlike some alternatives
Backend hosting	Render (free web service)	750 hrs/mo, one instance	Sleeps after 15 min idle; ~30–60s cold start on the next request
Frontend hosting	Vercel (Hobby)	100 GB bandwidth/mo, 1M function calls	ToS restricts Hobby to personal/non-commercial use — see note below
Frontend hosting (alt.)	Cloudflare Pages	Unlimited static bandwidth, 500 builds/mo	Explicitly allows commercial use; Next.js needs the @cloudflare/next-on-pages adapter
Cache / rate limits	Upstash (Redis)	256 MB, 500K commands/mo	Idle databases can be reclaimed — a small scheduled ping keeps it alive
Object storage (Phase 2)	Cloudflare R2	10 GB storage, zero egress fees, ever	Free tier applies to Standard storage only
Transactional email	Resend	3,000 emails/mo, 100/day cap	SendGrid's old free tier (common in older tutorials) no longer exists
Bot protection	Cloudflare Turnstile	Unlimited, free	—
CDN / DDoS / DNS / TLS	Cloudflare (free plan)	Unmetered bandwidth on the free plan	—
Search (Phase 2)	Meilisearch, self-hosted	Free, open source (MIT), no feature cap	You host the small binary yourself — e.g. on the same free backend instance
CI / CD	GitHub Actions	2,000 min/mo private repos; unlimited on public repos	—
Error tracking	Sentry (free developer tier)	Limited monthly events	Fine for MVP traffic; re-check the current cap before relying on it

THE ONE CAVEAT WORTH READING TWICE

The single most important line in this table: **Vercel's Hobby plan is contractually for personal, non-commercial projects** — a real, public platform with users technically falls outside those terms even before it earns a cent. Cloudflare Pages' free tier explicitly permits commercial use and costs nothing, at the price of a small adapter and a few Workers-runtime constraints for Next.js. If you intend to run this for real, start there instead of Vercel, or budget for Vercel Pro ($20/month) once the ambiguity stops being comfortable.

NOTE

A second pattern shows up across three different free tiers here: Neon, Upstash, and (less aggressively) Render all reclaim or pause resources after a period with no traffic. A single free, scheduled GitHub Actions workflow that pings your backend, database, and Redis every few days closes this gap entirely and costs nothing — worth setting up in week one, not after the first time something has quietly gone to sleep.

15. DevOps & Deployment Pipeline

15.1 The whole pipeline, in one paragraph

Push to a `main` branch on GitHub. A GitHub Actions workflow (free) runs your test suite and linter on every push. If it passes, Render redeploys the backend and Vercel/Cloudflare Pages redeploys the frontend automatically — both platforms watch your repository directly, with no separate deploy script to write or maintain.

15.2 Environment variables and secrets

Database URLs, API keys, and the JWT signing secret live in each host's environment-variable settings, never in the codebase itself. A `.env` file holds them locally for development and is listed in `.gitignore` from the very first commit — a secret committed to git history is very difficult to fully remove later, so this is worth getting right on day one rather than fixing after the fact.

15.3 Keeping the free backend warm

Since Render's free tier sleeps after 15 minutes of inactivity (Section 14), a small free GitHub Actions workflow on a schedule (e.g. every 10 minutes) that pings a lightweight health-check endpoint keeps the service responsive for real visitors, at the cost of using part of your free CI minutes for something genuinely useful.

16. Legal & Trust Basics

> **BEFORE ANYTHING ELSE**
>
> I'm not a lawyer, and none of this is legal advice — it's a list of the things a platform like this typically needs to have someone look at, so you know what to raise with an actual lawyer before launch rather than after a problem.

- **A Privacy Policy and Terms of Service**, before real users sign up — not a nice-to-have, close to a baseline legal requirement almost everywhere, and also directly required by most of the free services in Section 14.
- **Data-protection law** if you have users in the EU (GDPR) or California (CCPA) — both include rights like data export and account deletion that are much easier to design in from the start than to retrofit onto an existing schema.
- **Elevated defamation/reputational risk specific to fact-checking.** Labelling a real claim — and by extension, whoever posted it — as “false” is a stronger statement than most social platforms make about user content, and getting it wrong carries more weight than a typical moderation mistake. The appeals path in Section 8.4 and a visible audit trail (Section 3.4) both help; a lawyer should still review your Terms of Service language around verification labels specifically before a public launch.

17. Testing

“Production ready” includes a test suite, not just working code — the two are different claims, and only one of them survives a change to the codebase six months from now. All of the following are free and open source:

Layer	Tool	Tests
Backend	pytest-django	Model logic, the routing decision in Section 7.2, permission checks
Frontend	Jest + React Testing Library	Components render correctly and respond to interaction
End-to-end	Playwright	A full flow works together — e.g. sign up, post a fact, see the pending badge

Wire the backend and frontend test suites into the GitHub Actions workflow from Section 15.1, so a broken change fails the pipeline before it ever reaches Render or Vercel/Cloudflare Pages.

18. MVP Build Roadmap

A concrete build order — each phase produces something you can actually run and click through before starting the next one.

Phase	Build
1	Project setup: Django + DRF backend, Next.js frontend, Neon database connected, deployed skeleton on Render + Vercel/Cloudflare Pages (Sections 1–2, 14)
2	Signup, login, email verification, Google OAuth (Section 4)
3	Post creation with the opinion/fact selector; a plain reverse-chronological feed; no verification logic yet — fact posts simply sit at status='pending' (Section 5.1)
4	Follow, like, comment (Section 9)
5	Reporting system and the threshold logic (Section 6)
6	The pipeline interface as a stub — a fake endpoint returning a random or rule-based confidence score — wired to the real routing logic (Section 7). This lets you build and test everything downstream before the real pipeline exists.
7	Human Review Panel in Django Admin (Section 8)
8	Search (Postgres full-text, Section 10) and notifications (Section 11)
9	Security pass (Section 12) and a load test with k6 (Section 13.2)
10	Public launch on the free infrastructure in Section 14, with monitoring (Sentry) and the keep-alive workflow (Section 15.3) in place

19. Future Roadmap (Post-MVP)

Explicitly out of scope for now, but worth knowing the free stack in this guide leaves room for all of it without a rebuild:

- **Images and video.** Cloudflare R2 (Section 14) is already the right free target when this happens — zero egress fees matter a lot once you're serving media.
- **The real fact-checking pipeline.** Slots into the exact contract defined in Section 7.1 with no changes needed anywhere else in the system.
- **An algorithmic “For You” feed.** The reverse-chronological feed in Section 9 is a deliberate MVP choice, not a permanent ceiling.
- **Bridging-style crowd rating**, in the spirit of X's Community Notes (Section 7.3) — worth revisiting once you have enough active users for cross-perspective agreement to be statistically meaningful.
- **Native mobile apps.** Because the backend is already a clean REST API rather than something tied to server-rendered pages, a mobile client can be added later as just another API consumer, with no backend changes required.

> **WHY THIS IS FREE**
>
> Closing note: every recommendation in this guide was chosen to be both genuinely free today and genuinely the same shape used at real scale — so the work you do building the MVP is work that still counts if this grows. Good luck with it.